

Bases, Structures & Logique

1. Squelette de Script & Fonctions

En-tête obligatoire (Shebang) :

```
#!/usr/bin/env python3
```

Cette ligne permet d'exécuter le script via `./script.py` après un `chmod +x`.

Structure "Main" (Bonne pratique) : Empêche l'exécution du code lors d'un import.

```
if __name__ == "__main__":  
    # Votre code principal ici  
    pass
```

Définition de Fonction :

```
def ma_fonction(param: int, opt: str = "défaut") -> bool:  
    """ Docstring : Description de la fonction """  
    return True
```

- Arguments par défaut toujours à la fin.
- Typage indicatif (`int`, `-> bool`) utile pour la lisibilité.

2. Structures de Données

Type	Syntaxe	Modifiable	Doublons	Accès / Notes
Liste	<code>l =</code>	OUI	OUI	<code>l.append(x)</code> , ordonné
Tuple	<code>t = (1, 2)</code>	NON	OUI	<code>t</code> , plus rapide, immuable
Set	<code>s = {1, 2}</code>	OUI	NON	Pas d'ordre. Opérations ensembles
Dict	<code>d = {"k":v}</code>	OUI	Clés:Non Clé -> Valeur. Ordonné (Py 3.7+)	

Manipulation des Sets (Ensembles) :

- Union: `s1 | s2` | Inter: `s1 & s2` | Diff: `s1 - s2`
- Ajout: `s.add(x)` | Suppr: `s.remove(x)` (erreur si absent), `s.discard(x)` (pas d'erreur).

Manipulation des Dictionnaires :

- Accès sûr: `d.get("clé", "valeur_défaut")` (évite le crash si clé absente).
- Parcours :
- Ajout/Modif: `d["clé"] = valeur` ou `d.update({"k": v})`.

3. Boucles & Random

Boucle While : À utiliser quand on ne connaît pas le nombre d'itérations à l'avance (ex: saisie utilisateur).

```
while True:  
    n = input("Saisie: ")  
    if condition_sortie:  
        break # Seule sortie possible
```

Module Random (`import random`):

- `random.randint(a, b)` : Entier aléatoire `N` tel que `a <= N <= b`.
- `random.choice(seq)` : Un élément au hasard dans une liste `seq`.
- `random.choices(seq, k=2)` : Liste de `k` éléments (avec remise).
- `random.sample(seq, k=2)` : Liste de `k` éléments uniques (sans remise).
- `random.shuffle(seq)` : Mélange la liste `seq` sur place (ne retourne rien).

Système, Fichiers, Web & Regex

4. Arguments de Script (Argparse)

Standard imposé pour les TP. Remplace `sys.argv`.

```
import argparse  
parser = argparse.ArgumentParser(description="Description du script")  
  
# Argument positionnel (Obligatoire)  
parser.add_argument("nom", help="Votre nom")  
  
# Argument optionnel (Positionnel avec valeur par défaut)  
parser.add_argument("n", type=int, nargs="?", default=5, help="Un entier")  
  
# Option (Flag ex: -v)  
parser.add_argument("-v", "--verbose", action="store_true", help="Mode verbeux")  
  
args = parser.parse_args()  
# Utilisation : print(args.nom, args.n)  
# Si -v est présent, args.verbose vaut True
```

5. Fichiers & JSON

Lecture / Écriture Fichier Texte :

- Mode : 'r' (lecture), 'w' (écriture/écrasement), 'a' (ajout).

Lecture

```
with open("fich.txt", "r") as f:
    # tout = f.read() # Tout le texte (str)
    lignes = f.read().splitlines() # Liste des lignes sans \n
```

Écriture

```
with open("fich.txt", "w") as f:
    f.write("Texte\n")
    f.writelines([str(x)+"\n" for x in liste]) # Écrit une liste ligne par ligne
```

JSON (import json) :

- `data = json.load(f)` : Lit un fichier JSON ouvert -> Dict/List Python.
- `json.dump(data, f)` : Écrit Dict/List Python -> fichier JSON ouvert.
- Conversion types : object ↔ dict, array ↔ list, null ↔ None.

6. Système (OS & Subprocess)

Parcours de dossiers (os.walk) :

```
import os
for racine, dossiers, fichiers in os.walk("."):
    for nom_fich in fichiers:
        chemin_complet = os.path.join(racine, nom_fich)
        if os.path.getsize(chemin_complet) == 0: # Taille nulle
            os.remove(chemin_complet) # Supprimer fichier
```

Commandes Système (subprocess) :

```
import subprocess
# shell=True pour pipes/jokers. capture_output pour récupérer le texte.
res = subprocess.run(["ping -c1 127.0.0.1"], shell=True, capture_output=True)
sortie = res.stdout.decode("utf-8") # Bytes -> Str
if "bytes from" in sortie:
    print("UP")
```

- OS Utiles : `os.makedirs("a/b")` (mkdir -p), `os.getcwd()` (pwd), `shutil.rmtree()` (rm -rf).

7. Web (Requests) & Regex (Re)

Requête HTTP (import requests) :

• Proxy IUT (Indispensable) :

```
proxies = { "http": "http://cache-etu.univ-artois.fr:3128",
            "https": "http://cache-etu.univ-artois.fr:3128" }
```

```
# r = requests.get(url, proxies=proxies)
```

Expressions Régulières (import re) :

- **Motifs** : . (tout), \d (chiffre), \w (alphanum), + (1 ou +), * (0 ou +), ? (0 ou 1), {n} (n fois).

- [abc] (a,b ou c), ^ (début), \$ (fin).

• Utilisation :

```
motif = re.compile(r"votre_regex")
for match in motif.finditer(texte):
    print(match.group()) # Affiche la chaîne trouvée
```

- **Exemple IP** : `r"\d{1,3}(\.\d{1,3}){3}"`.

- **Exemple Email simple** : `r"[\w\.-]+@[\w\.-]+"`.